

TENURE-11 ENGINEERING PACKET: GENERATIONAL COLLECTOR TUNING

1. PURPOSE AND SCOPE

A latency-bounded service runs one fixed allocation workload (Figure 4) on a generational garbage collector. The task is to simulate the collector exactly as described here for every candidate configuration, then choose settings. All quantities are bytes or integer pause units. All arithmetic is integer; every division rounds down. Nothing here depends on wall-clock time, randomness, or any external library.

2. OBJECTS AND SIZES

Every object has a kind (Figure 3). Its size is HEADER (Figure 2) plus its payload, rounded up to the next multiple of $\text{ALIGN} = 8$. Payload is the kind's base payload from Figure 3, plus, for BLOB, ENTRY and NODE, the variable term printed under Figure 3, where i is the index of the operation that allocates the object. Each kind has a fixed number of reference slots (dots in Figure 3), numbered from 0, each holding either null or a reference to one object.

3. ROOTS

The mutator holds references only in numbered root slots: slot 0 (scratch), slots 1 to 64 (session ring), slots 100 to 115 (registry), slot 120 (list head). All start null. Whenever roots are scanned, non-null root slots are visited in ascending slot number.

4. THE WORKLOAD

Before operation 0, the program allocates 16 REG objects one after another and puts the j -th ($j = 0$ to 15) into root slot $100 + j$. Then operations run for every index i from 0 up to, but not including, the end mark of the timeline in Figure 4. Each phase covers the operation indexes from its left edge up to, but not including, its right edge. Operation i allocates one object of the kind given by its phase's mix at position $(i \bmod 10)$, then acts on it:

TEMP: root slot 0 is set to the new object.

SESSION: if k sessions were created before this one, root slot $1 + (k \bmod 64)$ is set to the new session. The operation then allocates a BLOB (same i). After that allocation completes, it reads the session back from its root slot (a collection during the BLOB allocation may have relocated it) and stores the BLOB into the session's slot 0.

ENTRY: after the allocation, if any session exists, the entry's slot 0 is set to the object currently held in the ring slot of the most recently created session. If c entries were created before this one, the registry object in root slot $100 + (c \bmod 16)$ has its slot $(c \div 16) \bmod 4$ set to the entry.

NODE: after the allocation, the node's slot 0 is set to whatever root slot 120 currently holds, and root slot 120 is set to the node.

After its actions, if $i \bmod 1500$ equals 1499, the operation sets root slot 120 to null.

Every read the mutator performs happens after the allocation it follows, so it always sees current locations.

5. HEAP REGIONS (FIGURE 1)

The young generation is an eden of EDEN bytes and two survivor spaces of SURV bytes each; both sizes are configuration parameters. Eden and the survivor spaces are bump allocated from offset 0. At any time one survivor space (the from-space) holds the objects that survived the previous minor collection and the other (the to-space) is empty. Every young object carries an age, 0 when allocated. The old generation is OLD_SIZE bytes (Figure 1). It never moves objects. It keeps a free list of blocks (address, length), initially one block covering the whole old generation.

6. ALLOCATION

If the configuration's pretenure threshold PT is enabled and the object's size is at least PT, the object is placed directly in the old generation (section 8). If that placement fails, one major collection runs (section 9) and placement is attempted once more; if it fails again the run ends in OOM. Otherwise the object goes to eden: if eden's used bytes plus the object's size would exceed EDEN, a minor collection runs first (section 7); the object is then bump allocated in eden with age 0.

7. MINOR COLLECTION

Before anything else, a minor collection checks a promotion guard: if the total number of free bytes in the old generation is smaller than eden's used bytes plus the from-space's used bytes, a major collection (section 9) runs first, and is logged first. The minor collection then records R , the current number of objects in the remembered set.

Live young objects are found and moved in a fixed order. First every non-null root, in ascending slot order. Then every remembered-set object, in ascending old-generation address, and within each such object its slots in slot order. Then a first-in first-out queue holding every object moved so far in this collection, in the order they were moved, whether they went to the to-space or to the old generation: for each queued object, its slots in slot order. Whenever this procedure reaches a reference to a young object that has not yet been moved in this collection, that object is moved now and appended to the queue; a young object already moved in this collection is not moved again, and the reference is simply updated to its new location. References to old objects are left alone.

Where a moved object goes is decided by its new age, which is its old age plus one, and by the to-space's remaining room, as drawn in Figure 5. An object moved to the to-space is bump allocated there with its new age, and its size counts toward bytes copied. An object moved to the old generation is placed by section 8, and its size counts toward bytes promoted. If any such placement fails, the run ends in OOM.

Afterwards eden and the old from-space are empty, and the to-space becomes the from-space. The remembered set is rebuilt as exactly the old objects holding at least one reference to an object now in the survivor space. The pause of the minor collection is $40 + (\text{bytes copied} \div 64) + (\text{bytes promoted} \div 32) + 2 \times R$.

8. OLD-GENERATION PLACEMENT

Placement of an object of size s takes the free block with the lowest address whose length is at least s . If the block's length minus s is at least 32, the object occupies the first s bytes and the rest stays a free block; otherwise the object occupies the entire block. Each old object remembers how many bytes it occupies. Placement never merges free blocks.

9. MAJOR COLLECTION

A major collection marks every object reachable from the non-null roots, following every reference in every region. It then frees every unmarked old object: its occupied block is returned to the free list and it leaves the remembered set. Finally the free list is sorted by address and blocks that are exactly adjacent are merged. Young objects are not moved. The pause is $150 + (\text{total occupied bytes of the marked old objects} \div 32) + (\text{number of old objects freed} \div 2)$.

10. WRITE BARRIER

When the mutator stores a non-null reference into a slot of an object that is in the old generation, and the referenced object is in eden or a survivor space, the old object is added to the remembered set. Adding an object already in the set has no effect. Stores that do not meet both conditions do not touch the set.

11. METRICS PER CONFIGURATION

oom (and the index of the operation during which it happened; registry setup counts as operation -1), number of minor collections, number of major collections, bytes copied, bytes promoted, bytes pretenured (sizes of objects placed directly by section 6), total pause (sum of all minor and major pauses), maximum pause, and peak old occupancy. Old occupancy is the sum of occupied bytes of all old objects; the peak is the largest value seen right after each minor collection and right after each pretenured placement. If a run ends in OOM, the configuration is infeasible.

12. CANDIDATE CONFIGURATIONS AND SELECTIONS

Every combination of EDEN in {16384, 24576, 32768, 49152}, SURV in {8192, 24576}, T in {1, 2, 3, 5} and PT in {off, 768, 1024} is a candidate, 96 in all, each simulated as a fresh, complete run of the workload. Configurations are ordered by EDEN, then SURV, then T, then PT (off before 768 before 1024). Over feasible configurations only:

Cost optimal: smallest total pause; ties by smallest maximum pause, then configuration order.

Latency optimal under budget: among configurations with total pause at most 220000, smallest maximum pause; ties by smallest total pause, then configuration order.

Footprint optimal under ceiling: among configurations with maximum pause at most 2400, smallest EDEN + 2 x SURV; ties by smallest total pause, then configuration order.

13. BASELINE AND ITS EVENT LOG

The service currently runs EDEN 24576, SURV 8192, T 2, PT off. For this baseline, produce the GC event log in the format of Figure 6, one line per minor or major collection in the order they happen. Field op is the index of the operation during which the collection happened (registry setup counts as -1). For a minor collection, remset is R , marked and freed are 0, and copied and promoted are its own bytes. For a major collection, copied, promoted and remset are 0, marked is the occupied bytes of the marked old objects, and freed is the number of old objects freed. Occupancy is old occupancy immediately after the event. Pause is that event's pause.

Heap map. OLD_SIZE is the full span of the old-generation bar.

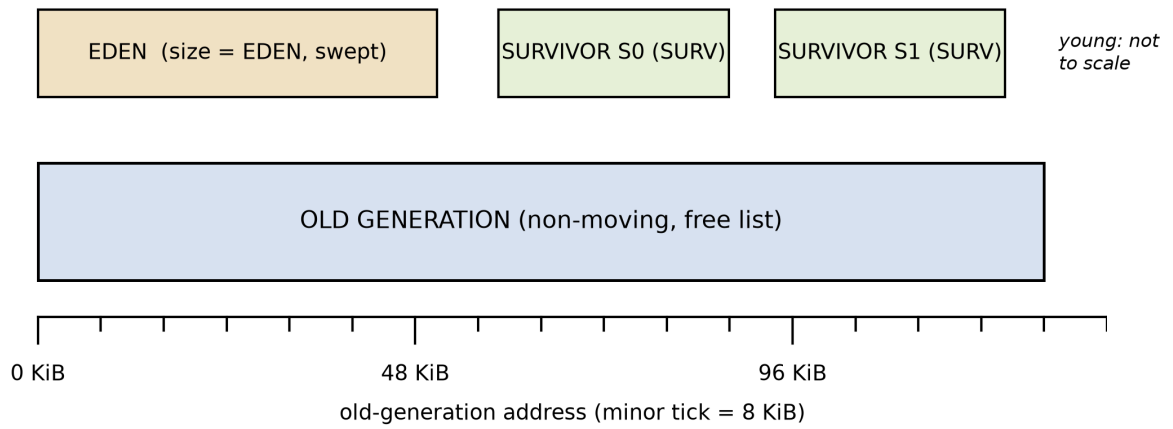


Figure 1 -- Heap map.

Object layout. Every object starts with a HEADER of this length.

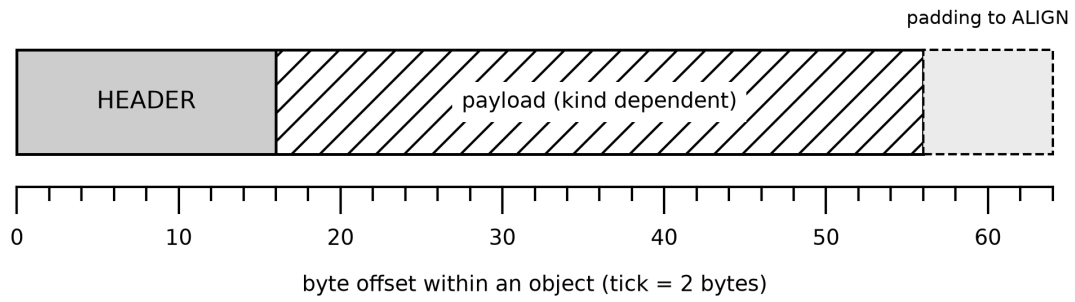
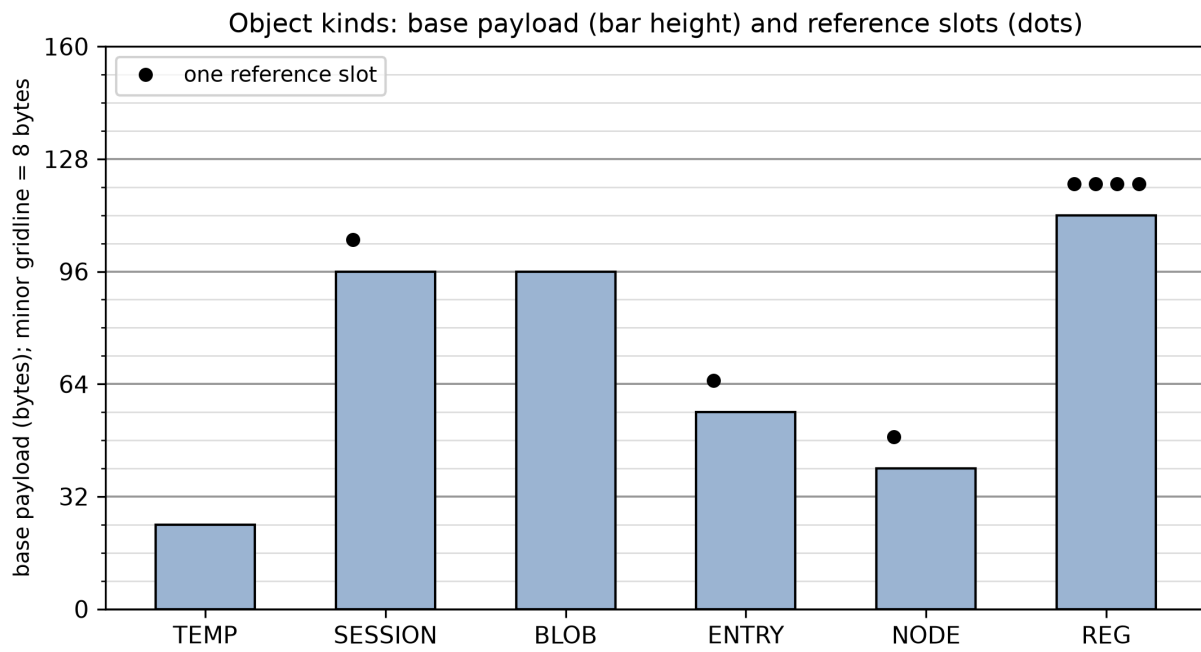


Figure 2 -- Object layout.



Size-varying kinds (i = index of the allocating operation):
 BLOB payload = base + $96 \times (i \bmod 13)$ ENTRY payload = base + $24 \times (i \bmod 3)$
 NODE payload = base + $8 \times (i \bmod 5)$ all other kinds: payload = base

Figure 3 -- Object kinds.

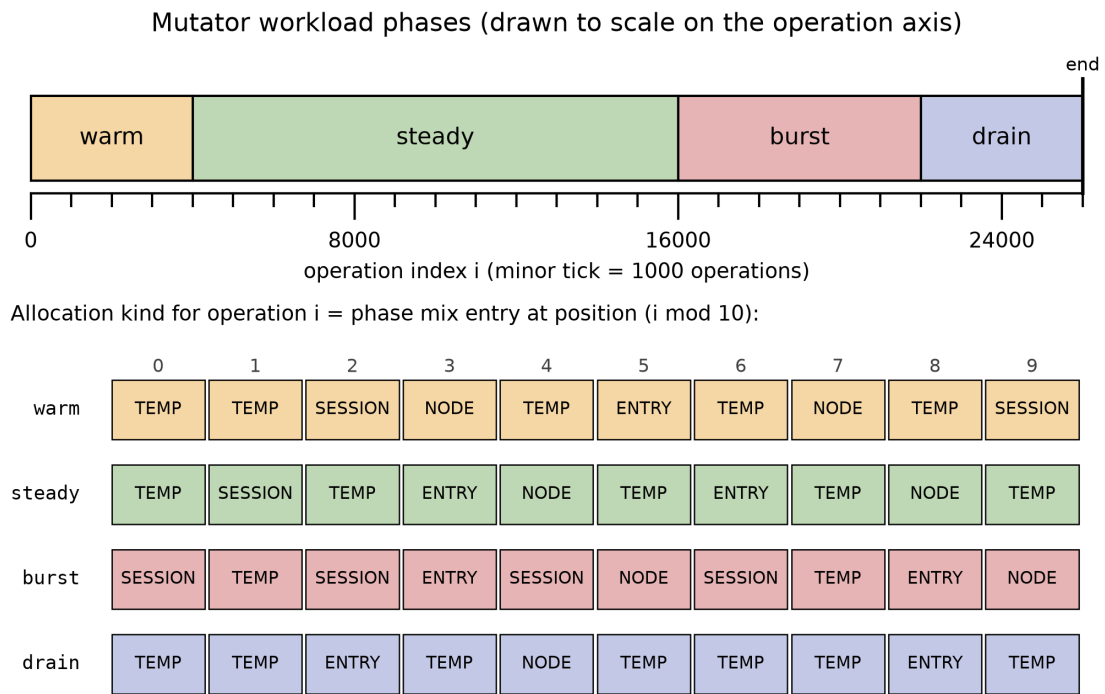
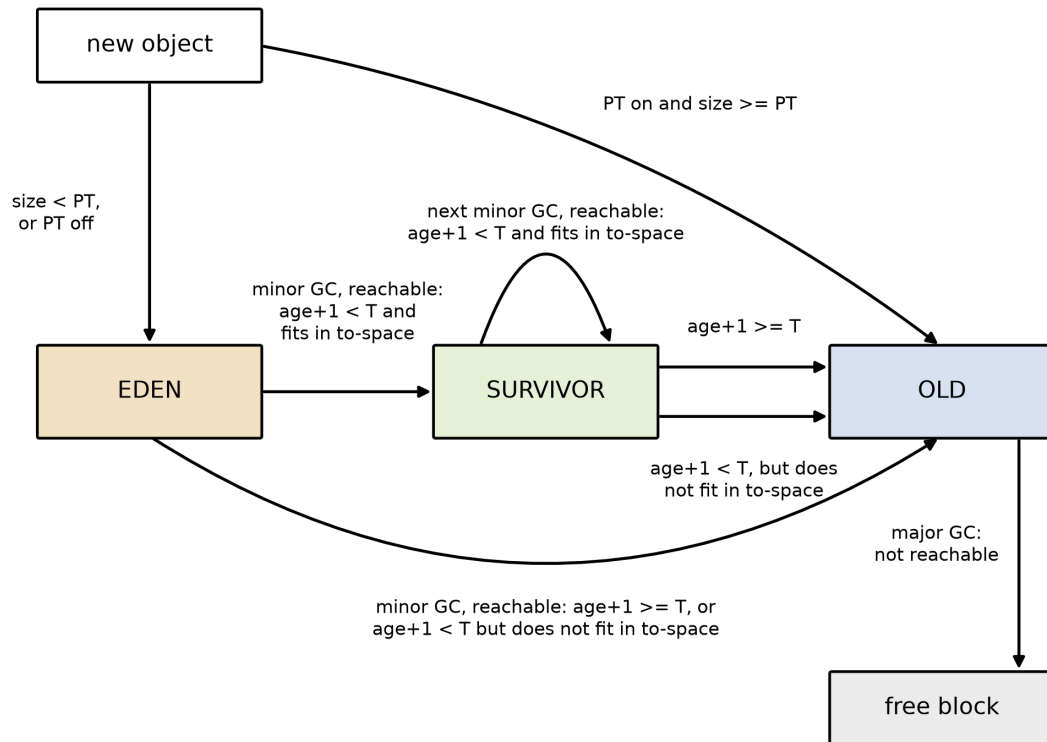


Figure 4 -- Mutator workload.

Object lifecycle (T = tenuring threshold, PT = pretenure threshold)



Reachable means reached by the minor collection's evacuation order. Unreachable young objects are abandoned when eden and the from-space are emptied.

Figure 5 -- Object lifecycle.

Baseline GC event log (certificate) format

GC event log format, worked example with FAKE numbers. The values below do not come from this model and need not satisfy its rules; they show the TEXT FORMAT only.

TENURE11-GCLOG-V1

```
1;m;37;5104;912;0;0;0;912;151
2;M;58;0;0;0;3320;7;2208;360
3;m;58;4488;1376;3;0;0;3584;157
...
```

Fields: n;type;op;copied;promoted;remset;marked;freed;occupancy;pause
One line per GC event in the order the events happen; integers in plain decimal, no padding, no spaces; type is m (minor) or M (major); fields that do not apply to the type are 0. Lines are joined with a single newline and the text ends with one trailing newline. The log hash is the first 16 lowercase hex characters of SHA-256 of that text.

Figure 6 -- GC event log format (fake example).